

Architetture Software per la Robotica Autonoma: Il Middleware ROS 2

NECKER

15 Giugno 2026

Indice

1	Introduzione all'Architettura Software di un Robot	2
1.1	Scomposizione Hardware-Software: Studio di Caso TurtleBot3	2
1.2	Il Concetto di Middleware ed Astrazione	3
2	Il Framework ROS 2 (Robot Operating System)	3
2.1	Distribuzioni e Ciclo di Rilascio di ROS 2	5
3	Il Grafo Computazionale di ROS 2	5
3.1	Le Interfacce e il linguaggio IDL	5
3.2	I Nodi (Nodes)	5
3.3	I Nodi Gestiti (Managed Lifecycles)	6
3.4	I Topic: Meccanismo Publish/Subscribe	7
3.5	I Servizi: Struttura Request/Response	7
3.6	Le Azioni: Struttura basata su Goal, Feedback e Result	7
3.7	I Parametri (Parameters)	7
3.8	Anatomia dei Messaggi Sensoriali Comuni	8
3.8.1	LaserScan (<code>sensor_msgs/msg/LaserScan</code>)	8
3.8.2	Immagini (<code>sensor_msgs/msg/Image</code>)	8
4	Il Sottosistema TF2 (Trasformazioni Spaziali)	9
5	Il Middleware DDS e la Qualità del Servizio (QoS)	10
5.1	Architettura Decentralizzata DDS	10
5.2	I Profili della Qualità del Servizio (QoS - Quality of Service)	10
6	I Behavior Trees	11
6.1	Meccanismo del "Tick" e Stati di Ritorno	11
6.2	Tassonomia dei Nodi nei Behavior Trees	11
6.2.1	Nodi di Controllo (Control Nodes)	11
6.2.2	Nodi Decoratori (Decorator Nodes)	11
6.2.3	Nodi Foglia (Leaf Nodes)	12
6.3	Analisi di un Workflow Decisionale Avanzato	12
6.3.1	Analisi del ciclo di esecuzione (<i>Trace dell'Albero</i>)	13

1 Introduzione all'Architettura Software di un Robot

Dall'analisi ingegneristica e informatica, un robot autonomo non può essere considerato come un semplice computer, bensì come un sistema cyber-fisico complesso integrato, equipaggiato con sensori (per la percezione dell'ambiente) e attuatori (per l'interazione fisica). La progettazione del software di controllo richiede la coesistenza di task puramente computazionali ad alta intensità algoritmica con vincoli stringenti di tempo reale, determinismo e reattività agli stimoli ambientali.

1.1 Scomposizione Hardware-Software: Studio di Caso TurtleBot3

Per comprendere la necessità di un'architettura software modulare e stratificata, è utile analizzare la scomposizione dei compiti all'interno di una piattaforma di riferimento industriale e accademica come il **TurtleBot3 Burger**. La gestione delle risorse computazionali è distribuita su una topologia hardware eterogenea, divisa principalmente tra elaborazione ad alto livello ed esecuzione a basso livello e tempo reale:

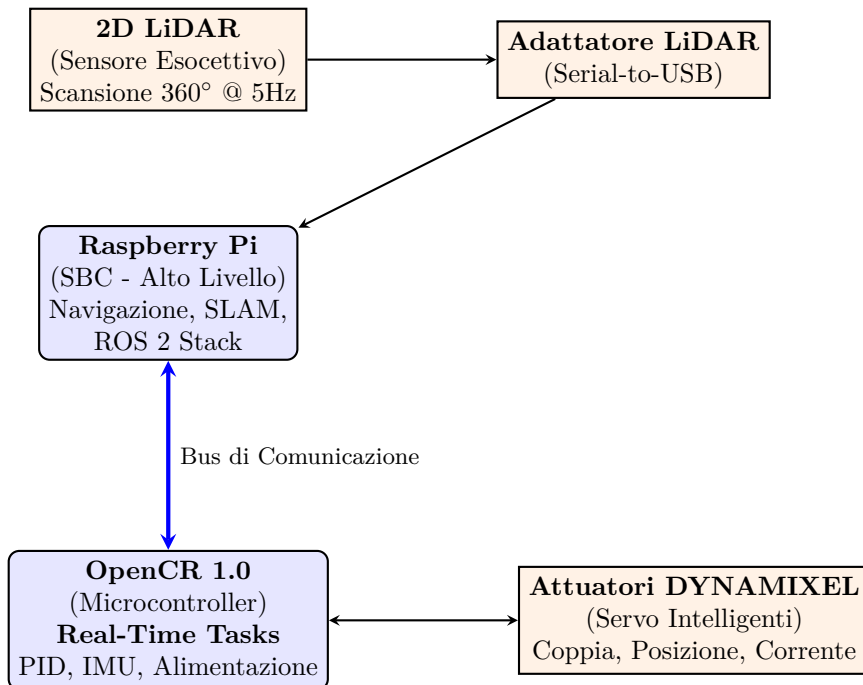


Figura 1: Topologia hardware e partizionamento computazionale del TurtleBot3 Burger.

- **Single-Board Computer (Raspberry Pi):** Dedicato a task ad alta intensità di calcolo e non strettamente vincolati dal tempo reale deterministico. Gestisce l'elaborazione delle nuvole di punti del LiDAR, l'esecuzione degli algoritmi di *Simultaneous Localization and Mapping* (SLAM), la pianificazione globale e locale delle traiettorie (*MoveBase/Nav2*) e l'orchestrazione dei nodi del middleware.
- **Microcontrollore Specializzato (OpenCR 1.0):** Una scheda basata su architettura ARM Cortex-M7 progettata specificamente per ROS. Gestisce task a bassissima latenza che richiedono determinismo temporale, tra cui: il ciclo di controllo PID per la velocità dei motori, la lettura dei dati grezzi dell'Unità di Misura Inerziale (IMU a 6 assi con giroscopio e accelerometro), la diagnostica dello stato della batteria e l'interfaccia hardware diretta.

- **Sensore 2D LiDAR:** Sensore esocettivo che esegue una scansione planare continua a 5Hz in un raggio di 360° fino a una distanza di 3.5 metri, convertendo i dati seriali grezzi tramite un adattatore USB per renderli accessibili al Single-Board Computer.
- **Attuatori DYNAMIXEL:** Servomotori intelligenti dotati di microprocessore interno, in grado di restituire al microcontrollore feedback precisi su posizione angolare, velocità, temperatura e carico di corrente.

1.2 Il Concetto di Middleware ed Astrazione

Per evitare che lo sviluppatore debba riscrivere i driver di basso livello e gestire manualmente la sincronizzazione dei thread e la serializzazione dei pacchetti di rete, l'architettura software moderna introduce uno strato di **Middleware**. Il middleware si posiziona come un'astrazione software sopra il sistema operativo e l'hardware, standardizzando l'accesso ai sensori/attuatori e formalizzando i canali di scambio dati tra processi distinti tramite un paradigma guidato dai dati (*data-centric*).

2 Il Framework ROS 2 (Robot Operating System)

ROS 2 rappresenta l'evoluzione radicale di ROS 1, riprogettato da zero per soddisfare requisiti industriali che il framework originale non poteva garantire nativamente, quali: il supporto a sistemi embedded *bare-metal*, la conformità al tempo reale (*Real-Time compliance*), la tolleranza a reti wireless non affidabili o a banda limitata, la sicurezza crittografica del canale dati e le architetture multi-robot.

L'architettura interna di ROS 2 è strutturata in uno stack a strati rigidi, progettati per massimizzare la portabilità e disaccoppiare l'interfaccia applicativa dal trasporto fisico:

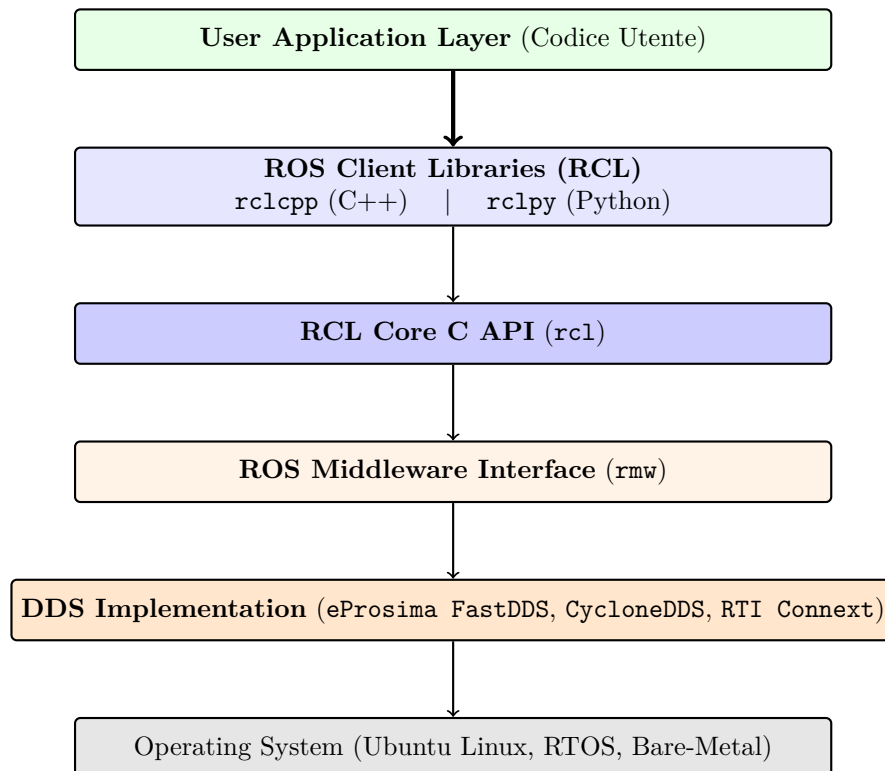


Figura 2: Architettura stratificata interna del framework ROS 2.

- **User Application Layer (Codice Utente):** È il livello più alto e rappresenta il codice sorgente scritto dallo sviluppatore. In questo strato risiede la logica specifica del robot, come gli algoritmi di navigazione, il controllo dei motori o l'elaborazione delle immagini, implementati sfruttando i concetti astratti (nodi, topic, servizi) messi a disposizione dal framework.
- **ROS Client Libraries (RCL):** Sono le librerie front-end specifiche per il linguaggio di programmazione scelto dall'utente. Le più comuni sono `rclcpp` per il C++ e `rclpy` per Python. Forniscono un'interfaccia ad alto livello (API) ricca e idiomatica per interagire con l'ecosistema ROS 2, rendendo lo sviluppo più intuitivo.
- **RCL Core C API (rcl):** È il cuore comune del framework, scritto in puro linguaggio C. Implementa la logica fondamentale di ROS 2 in modo neutrale rispetto al linguaggio di programmazione. Questo livello garantisce che il comportamento del sistema (come la gestione dei timer o il ciclo di esecuzione) rimanga coerente e identico, sia che l'utente stia programmando in Python, C++ o Rust.
- **ROS Middleware Interface (rmw):** È un livello di astrazione cruciale (*wrapper*) che separa la logica di ROS 2 dall'implementazione della rete sottostante. Grazie a questa interfaccia, ROS 2 è agnostico rispetto al middleware: permette all'utente di cambiare il fornitore DDS sottostante senza dover modificare una singola riga del codice applicativo.
- **DDS Implementation:** Rappresenta l'implementazione effettiva del protocollo di rete standard *Data Distribution Service* (DDS). Questo strato gestisce il "lavoro sporco" della comunicazione di rete: la scoperta automatica dei nodi (*discovery* peer-to-peer), la serializzazione/deserializzazione dei dati e l'applicazione delle policy di Qualità del Servizio (QoS). L'utente può scegliere tra vari fornitori supportati, come *eProsima FastDDS*, *Eclipse CycloneDDS* o *RTI Connex*.
- **Operating System:** È il sistema operativo host su cui viene eseguito l'intero stack. Oltre al classico supporto per Ubuntu Linux (standard de facto in robotica), l'architettura modulare di ROS 2 e l'uso del DDS consentono l'esecuzione nativa anche su macOS, Windows, sistemi operativi in tempo reale (RTOS) e piattaforme embedded *Bare-Metal* prive di sistema operativo tradizionale.

Come possiamo vedere, **ROS2** viene considerato un **middleware** e non un programma per varie ragioni come:

- Non è un singolo programma ma ogni nodo è un programma diverso in modo da non far crashare tutto il sistema insieme nel caso qualcosa fallisse.
- **ROS2** è legato ad una specifica versione di linux a causa delle sue dipendenze alle librerie di **glibc** che cambiano ad ogni release di linux.
- Un programma generalmente non integra tutto il sistema di gestione dei pacchetti, cosa che **ROS2** fa.

2.1 Distribuzioni e Ciclo di Rilascio di ROS 2

A differenza dei software tradizionali, ROS viene distribuito in pacchetti tematici chiamati "Distribuzioni" (*Distributions*). Ogni distribuzione garantisce che un set specifico di moduli e librerie sia compatibile e testato per lavorare congiuntamente.

I nomi delle distribuzioni seguono un ordine alfabetico e sono composti da un aggettivo e dal nome di una specie di tartaruga (es. *Ardent Apalone*, *Bouncy Bolson*, *Humble Hawksbill*). Una caratteristica fondamentale dell'architettura di ROS 2 è che ogni distribuzione è strettamente legata a una specifica release del sistema operativo di base, tipicamente Ubuntu Linux. Ad esempio, la distribuzione **Humble Hawksbill** (rilasciata a maggio 2022) è ancorata a Ubuntu 22.04 LTS. Questo legame garantisce stabilità e previene conflitti con le librerie di sistema. Inoltre, esiste una distribuzione speciale chiamata **Rolling Ridley**, costantemente aggiornata, che funge da area di test (*staging*) per le future release stabili.

3 Il Grafo Computazionale di ROS 2

Il Grafo Computazionale è la rete peer-to-peer di processi robotici che elaborano congiuntamente i dati. Gli elementi atomici che compongono questo grafo sono strutturati secondo regole di comunicazione formali.

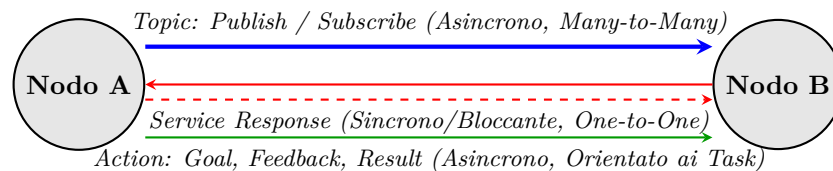


Figura 3: I tre paradigmi di comunicazione principali nel Grafo Computazionale di ROS 2.

3.1 Le Interfacce e il linguaggio IDL

Affinché i nodi possano interagire scambiandosi dati, ROS 2 impone l'uso di **Interfacce** rigorose. Un'interfaccia funge da "contratto" formale che assicura che il nodo mittente e il nodo destinatario concordino perfettamente sulla struttura e sul tipo di dati scambiati. Questi contratti sono scritti in un linguaggio neutro chiamato **IDL (Interface Definition Language)**, il quale permette a nodi sviluppati in linguaggi di programmazione differenti (es. C++ e Python) di comunicare senza alcun problema di compatibilità. A livello di file system, le interfacce si dividono in tre estensioni: `.msg` per i messaggi dei Topic, `.srv` per le richieste/risposte dei Servizi, e `.action` per i task complessi delle Azioni.

3.2 I Nodi (Nodes)

Un **Nodo** è l'unità computazionale elementare, responsabile di un compito specifico e isolato (es. controllo motori, filtraggio laser). In ROS 2, la scrittura dei nodi segue rigorosamente il paradigma della programmazione orientata agli oggetti (OOP). Un nodo viene implementato ereditando dalla classe base `rclcpp::Node`, incapsulando al suo interno publisher, subscriber, service e timer come membri privati della classe, garantendo pulizia del codice e isolamento della memoria.

3.3 I Nodi Gestiti (Managed Lifecycles)

Nel lancio standard di un nodo, questo inizia a eseguire il proprio comportamento non appena viene avviato (metodo *spin*). Tuttavia, per sistemi di controllo complessi, questo approccio è limitante. Per ovviare al problema, ROS 2 ha introdotto i **Managed Lifecycles** (Cicli di vita gestiti).

I nodi "gestiti" attraversano una macchina a stati deterministica composta da quattro stati principali: **Unconfigured**, **Inactive**, **Active** e **Finalized**. Questo paradigma permette un avvio (*bringup*) estremamente affidabile del robot: si garantisce che tutte le dipendenze critiche e le interfacce hardware siano caricate e inizializzate correttamente prima che il nodo inizi a processare i dati operativi. Un supervisore esterno può mettere in pausa, riavviare o spegnere un singolo nodo a tempo di esecuzione senza dover far crollare o riavviare l'intero grafo computazionale.

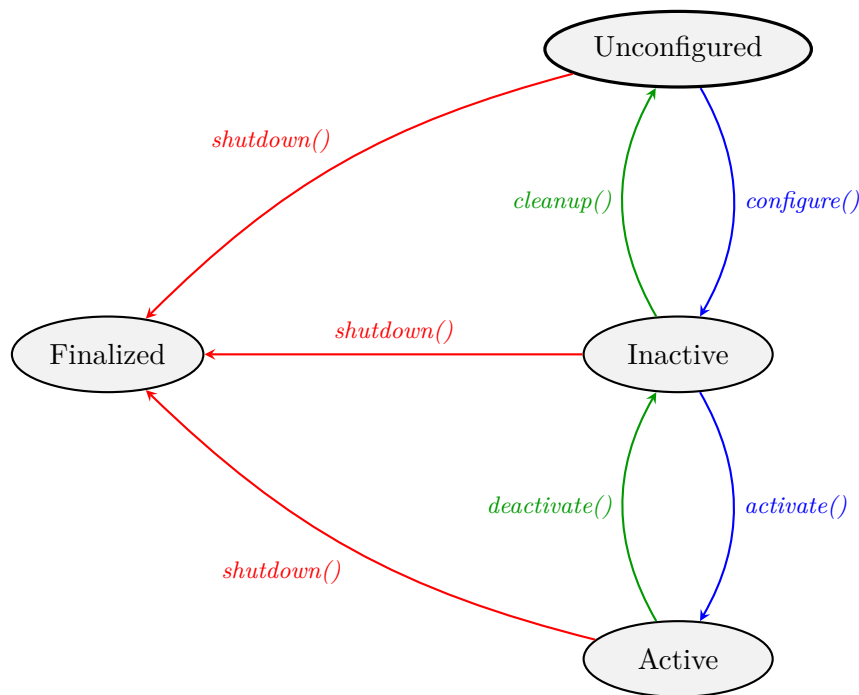


Figura 4: Macchina a stati semplificata del Lifecycle Node in ROS 2. Le transizioni permettono il controllo deterministico del ciclo di vita.

ATTENZIONE: lo stato **Finalized** corrisponde alla morte del nodo, infatti se per disattivarlo si passasse semplicemente ad **Unconfigured**, il nodo sarebbe ancora attivo in RAM. Facendo una operazione di shutdown invece, si attiva una routine per eliminare il nodo in sicurezza evitando di far rimanere per esempio un motore attivo poichè l'ultimo comando inviato non diceva di fermarsi.

3.4 I Topic: Meccanismo Publish/Subscribe

I **Topic** implementano un canale di comunicazione asincrono, anonimo e unidirezionale del tipo *many-to-many*.

- **Meccanismo:** Un nodo che produce dati li pubblica (*Publish*) su un determinato topic; uno o più nodi interessati a quel flusso di dati si iscrivono (*Subscribe*) al medesimo topic. I nodi non conoscono l'identità reciproca, conoscono solo il nome e il tipo del canale.
- **Casi d'uso ideali:** Flussi continui di dati da sensori a frequenza costante, dove la perdita sporadica di un pacchetto è preferibile al blocco del sistema (es. letture LiDAR, coordinate di odometria, immagini provenienti da una telecamera).

3.5 I Servizi: Struttura Request/Response

I **Servizi** (*Services*) definiscono una modalità di comunicazione bidirezionale, *one-to-one* del tipo client-server.

- **Meccanismo:** Un nodo Server offre una determinata funzionalità. Un nodo Client invia una *Request* (richiesta dati) e attende in modo sincrono o asincrono bloccante fino a quando il server non elabora il calcolo e restituisce una *Response* (risposta).
- **Casi d'uso ideali:** Operazioni estemporanee, computazioni matematiche repentine, comandi di reset o richieste di cambio di configurazione (es. scattare una fotografia ad alta risoluzione, azzerare l'odometria, richiedere la mappa statica dell'ambiente).

3.6 Le Azioni: Struttura basata su Goal, Feedback e Result

Le **Azioni** (*Actions*) rappresentano il paradigma di comunicazione più complesso di ROS 2, progettato specificamente per coordinare task a lungo termine ed evolutivi. Un'azione si appoggia su una struttura client-server asincrona non bloccante che implementa internamente tre canali di comunicazione distinti (creati combinando topic e servizi sottostanti):

1. **Goal Channel (Service):** Il client invia la richiesta di avvio di un compito complesso (es. "naviga fino al punto *X*"). Il server accetta o rifiuta il goal.
2. **Feedback Channel (Topic):** Durante l'esecuzione del task, il server invia aggiornamenti periodici asincroni al client per monitorare l'andamento (es. "distanza rimanente dal target: 1.2 metri").
3. **Result Channel (Service):** Al termine dell'esecuzione, il server notifica lo stato finale dell'operazione (successo, fallimento, abortito) inviando i dati consuntivi.

Un vantaggio fondamentale delle azioni è la **preemptibilità**: il client ha la facoltà di inviare un comando di cancellazione (*Cancel Goal*) in qualsiasi momento durante l'esecuzione se le condizioni esterne lo richiedono, interrompendo il task in sicurezza.

3.7 I Parametri (Parameters)

I **Parametri** in ROS 2 sono variabili di configurazione strutturate come coppie chiave-valore (es. `max_velocity = 0.22`) associate a un singolo nodo. Essi permettono di modificare il comportamento dinamico e algoritmico del software a tempo di esecuzione (*Runtime*) senza dover ricompilare il codice sorgente, esponendo un'interfaccia di riconfigurazione dinamica.

3.8 Anatomia dei Messaggi Sensoriali Comuni

Per standardizzare l'elaborazione dei dati ambientali, ROS 2 fornisce pacchetti contenenti definizioni pre-costruite dei messaggi. Due dei più utilizzati nella navigazione autonoma sono il LiDAR e la telecamera.

3.8.1 LaserScan (`sensor_msgs/msg/LaserScan`)

Rappresenta una scansione piana effettuata da un telemetro laser. Il messaggio è composto da:

- **Header:** Contiene il `timestamp` (istante di acquisizione) e il `frame_id` (il sistema di riferimento del sensore).
- **Geometria Angolare:** I campi `angle_min` e `angle_max` definiscono l'apertura in radianti della scansione (es. da 0 a 2π per una copertura a 360°). Il campo `angle_increment` indica la risoluzione angolare tra due misurazioni contigue.
- **Distanze (`ranges`):** Un array dinamico in virgola mobile che contiene le misurazioni fisiche della distanza in metri per ogni angolo scandito. Valori al di fuori dei limiti hardware (`range_min` e `range_max`) vengono scartati.
- **Intensità (`intensities`):** Indica la forza del raggio riflesso, utile per capire la riflettività o il materiale dell'ostacolo colpito.

3.8.2 Immagini (`sensor_msgs/msg/Image`)

Viene utilizzato per trasmettere matrici di pixel non compresse .

- I campi `height` e `width` definiscono le dimensioni spaziali dell'immagine in righe e colonne.
- Il campo `encoding` è una stringa fondamentale che spiega al Subscriber come interpretare i dati grezzi (es. `rgb8` per immagini a colori a 8-bit, o `32FC1` per mappe di profondità fluttuanti).
- L'array `data` contiene la sequenza ininterrotta dei byte dell'immagine.

A causa della loro dimensione, l'invio continuo di immagini non compresse satura rapidamente la banda di rete. Per questo motivo, ROS 2 utilizza un sistema a plugin chiamato **`image_transport`**, il quale permette di pubblicare le immagini applicando algoritmi di compressione (come JPEG, PNG o codec video Theora) sfruttando le differenze tra frame consecutivi.

4 Il Sottosistema TF2 (Trasformazioni Spaziali)

Un robot è fisicamente composto da molteplici parti mobili e sensori montati in posizioni asimmetriche. ROS 2 gestisce le relazioni geometriche tra tutte queste parti attraverso un approccio unificato chiamato sottosistema **TF2**.

TF2 tiene traccia nel tempo delle coordinate omogenee e delle matrici di rototraslazione (trasformazioni) che legano i vari sistemi di riferimento (chiamati *frames*). Il sistema adotta convenzioni fisiche molto rigide:

- Segue la **regola della mano destra** (Right-handed) per definire gli assi X, Y, Z .
- Gli angoli di rotazione crescono in senso antiorario (CCW): lo zero (0) punta in avanti, $\frac{\pi}{2}$ punta a sinistra, $-\frac{\pi}{2}$ punta a destra e $\pm\pi$ punta all'indietro.

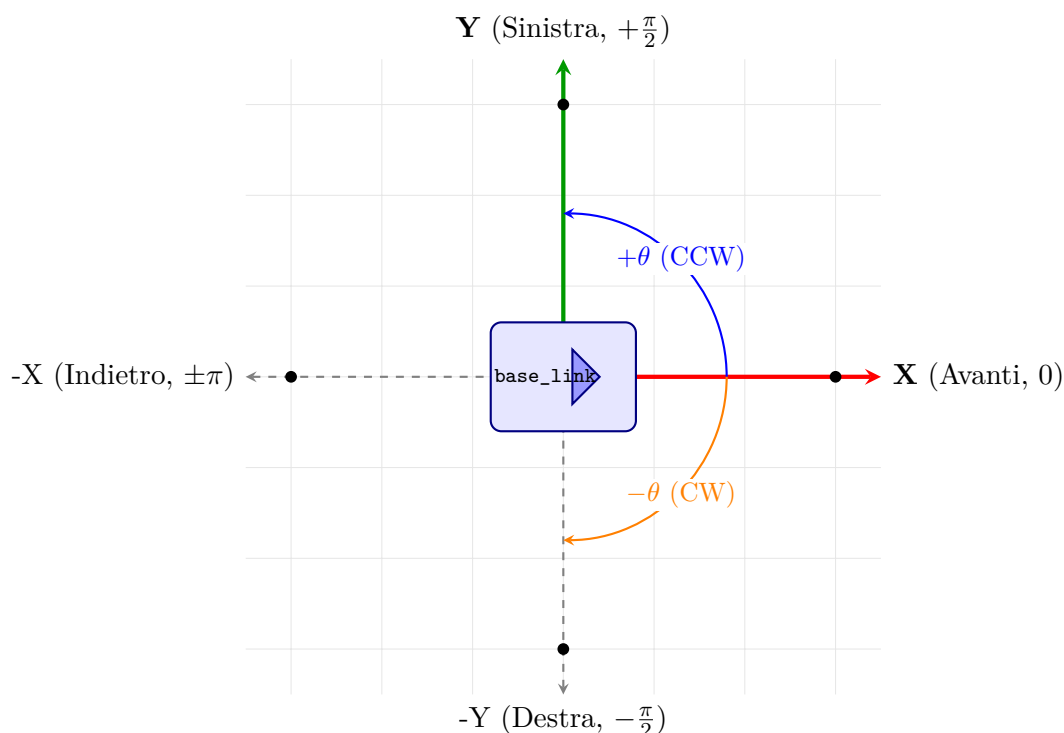


Figura 5: Convenzione spaziale standard di ROS 2 (TF2) basata sulla regola della mano destra. La vista è dall'alto (*Top-Down*). L'asse X indica il fronte del robot, l'asse Y il fianco sinistro. Le rotazioni avvengono sull'asse Z (uscente dal foglio) e crescono in senso antiorario.

Questo albero di trasformazioni si appoggia su due topic architetturali per gestire la diversità del mondo fisico:

1. **/tf**: Utilizzato per le trasformazioni dinamiche con alta volatilità temporale, come le ruote in movimento o i giunti di un braccio robotico. Questi dati scadono rapidamente se non vengono costantemente aggiornati.
ATTENZIONE: le **TF**, sono sempre espresse rispetto al sistema di riferimento del **parent frame** e non del **base_link**
2. **/tf_static**: Riservato per le trasformazioni rigide e immutabili, ad esempio la posizione in cui il sensore LiDAR è avvitato rispetto al centro geometrico del robot (**base_link**). Non avendo scadenza temporale, queste informazioni vengono trasmesse una sola volta sulla rete adottando una Qualità del Servizio (QoS) di tipo *Transient Local*.

5 Il Middleware DDS e la Qualità del Servizio (QoS)

A differenza di **ROS1**, in **ROS2** si è rimosso il ROS **Master** centralizzato a favore dell'adozione dello standard industriale **DDS (Data Distribution Service)**.

5.1 Architettura Decentralizzata DDS

Il DDS è un protocollo di comunicazione *data-centric* standardizzato dall'OMG (*Object Management Group*), ampiamente utilizzato nei settori aerospaziale, militare e dell'automazione industriale. Grazie al DDS, ROS 2 diventa un sistema totalmente decentralizzato e peer-to-peer: ogni nodo è in grado di scoprire la presenza di altri nodi sulla stessa rete locale in modo autonomo tramite pacchetti di **Discovery UDP**, eliminando il problema del *Single Point of Failure* rappresentato dal vecchio Master di ROS 1.

5.2 I Profili della Qualità del Servizio (QoS - Quality of Service)

Il DDS permette di configurare finemente le modalità di trasmissione dei messaggi a livello di trasporto di rete attraverso le policy di **Qualità del Servizio (QoS)**. La corretta impostazione di queste regole previene la congestione della banda e garantisce la robustezza del sistema. Le quattro policy fondamentali sono:

Policy QoS	Opzioni Disponibili	Comportamento Meccanico
History	- Keep Last (con profondità N) - Keep All	Keep Last conserva in memoria solo gli ultimi N messaggi ricevuti. Keep All conserva ogni messaggio finché la memoria non si esaurisce.
Reliability	- Reliable - Best Effort	Reliable implementa un controllo d'errore (ACK/NACK): se un pacchetto viene perso, viene reinviato. Best Effort invia i dati senza conferma, tollerando perdite.
Durability	- Volatile - Transient Local	Volatile cancella i messaggi passati; i nuovi iscritti vedono solo i messaggi futuri. Transient Local impone al publisher di conservare gli ultimi messaggi per inviarli ai nodi che si iscrivono in ritardo.
Liveliness	- Automatic - Manual By Node	Definisce le regole con cui un nodo dichiara alla rete di essere ancora attivo e funzionante (<i>heartbeat</i>).

Tabella 1: Sintesi delle policy QoS fondamentali nel middleware ROS 2 basato su DDS.

Alcuni esempi su come usarle:

- **Configurazione per Sensori ad Alta Frequenza (LiDAR, Cam):** Richiede **Reliability: Best Effort** e **History: Keep Last (depth=1)**. Non è utile sovraccaricare la rete per recuperare una scansione laser vecchia, è prioritario disporre istantaneamente dell'ultima lettura prodotta.
- **Configurazione per Mappe Statiche o Coordinate Critiche:** Richiede **Reliability: Reliable** e **Durability: Transient Local**. Se un nodo di navigazione si avvia in ritardo, deve poter ricevere immediatamente l'intera mappa dell'ambiente pubblicata ore prima dal nodo cartografico.

6 I Behavior Trees

Nella robotica autonoma complessa, la pianificazione delle decisioni ad alto livello (*Action Selection*) richiede un'architettura software flessibile. Storicamente, questo compito veniva affidato alle **Macchine a Stati Finiti (FSM)**. Tuttavia, le FSM soffrono del problema dell'esplosione degli stati, della scarsa modularità e della rigidità manutentiva: aggiungere uno stato richiede di ridisegnare manualmente le transizioni verso tutti gli altri stati preesistenti. Per superare questi limiti, i moderni stack di navigazione (come *Nav2* in ROS 2) adottano i **Behavior Trees (BT)**.

6.1 Meccanismo del "Tick" e Stati di Ritorno

Un Behavior Tree è un grafo orientato aciclico strutturato ad albero gerarchico. L'esecuzione del controllo è guidata da un segnale periodico emesso dalla radice (*Root Node*) chiamato **Tick**. Il Tick si propaga dall'alto verso il basso e da sinistra a destra lungo i rami dell'albero.

Ogni volta che un nodo riceve un Tick, esegue il proprio codice interno e deve **istantaneamente** restituire al nodo padre uno di tre possibili stati di ritorno cognitivi:

- **SUCCESS**: Il nodo ha completato il suo compito con successo.
- **FAILURE**: Il nodo ha fallito l'esecuzione del compito affidatogli.
- **RUNNING**: Il compito è in fase di esecuzione ma richiede tempo per essere completato (tipico delle azioni fisiche del robot). Al prossimo ciclo, il Tick tornerà a interrogare questo nodo.

6.2 Tassonomia dei Nodi nei Behavior Trees

I nodi di un Behavior Tree si dividono rigorosamente in due macro-categorie: nodi interni (di controllo o decorazione) e nodi foglia (esecutori).

6.2.1 Nodi di Controllo (Control Nodes)

Hanno il compito di indirizzare il flusso del Tick verso i propri figli in base agli stati di ritorno ricevuti.

- **Sequence ($\rightarrow$)**: Esprime la logica and. Esegue i figli in ordine sequenziale da sinistra a destra. Se un figlio ritorna **SUCCESS**, la Sequence passa immediatamente al figlio successivo. Se un figlio ritorna **FAILURE**, la Sequence interrompe l'esecuzione di tutti i figli successivi e restituisce immediatamente **FAILURE** al proprio padre. Se un figlio ritorna **RUNNING**, la Sequence si blocca temporaneamente e restituisce **RUNNING**.
- **Fallback / Selector (?)**: Esprime la logica or (gestione degli errori o comportamenti alternativi). Esegue i figli da sinistra a destra. Se un figlio ritorna **FAILURE**, il Fallback non si arrende, bensì passa ad interrogarne il successivo per tentare un recupero. Se un figlio ritorna **SUCCESS**, il Fallback si ritiene soddisfatto, interrompe la scansione e restituisce **SUCCESS**. Se un figlio ritorna **RUNNING**, restituisce **RUNNING**.
- **Parallel ($\Rightarrow$)**: Invia il Tick a tutti i suoi figli contemporaneamente, gestendo l'esecuzione concorrente di più attività. Ritorna **SUCCESS** o **FAILURE** in base al fatto che una determinata percentuale o un numero prefissato di figli abbia completato l'esecuzione.

6.2.2 Nodi Decoratori (Decorator Nodes)

Possono avere **un solo e unico figlio**. Il loro scopo è alterare o filtrare lo stato di ritorno del figlio o condizionarne il Tick (es. **Inverter** inverte il successo in fallimento e viceversa; **RetryUntilSuccessful** riesegue il figlio N volte in caso di fallimento prima di arrendersi).

6.2.3 Nodi Foglia (Leaf Nodes)

Rappresentano l'interfaccia diretta tra la logica dell'albero e lo stack funzionale di ROS 2. Non possono avere figli.

- **Condition Node (Ellisse):** Ispeziona lo stato del sistema o l'ambiente (es. controlla il livello della batteria o la presenza di un ostacolo). Ritorna immediatamente **SUCCESS** o **FAILURE**, non può mai ritornare **RUNNING**.
- **Action Node (Rettangolo):** Comanda un'azione fisica o computazionale complessa prolungata nel tempo (es. muovere un motore, calcolare un percorso). Tipicamente restituisce **RUNNING** finché l'attività non è conclusa.

6.3 Analisi di un Workflow Decisionale Avanzato

Per comprendere l'eleganza dei Behavior Trees nell'integrazione di ROS 2, analizziamo il seguente schema gerarchico, il quale modella un ciclo continuo di pattugliamento con controllo di sicurezza della batteria e routine di ripristino (*Recovery*):

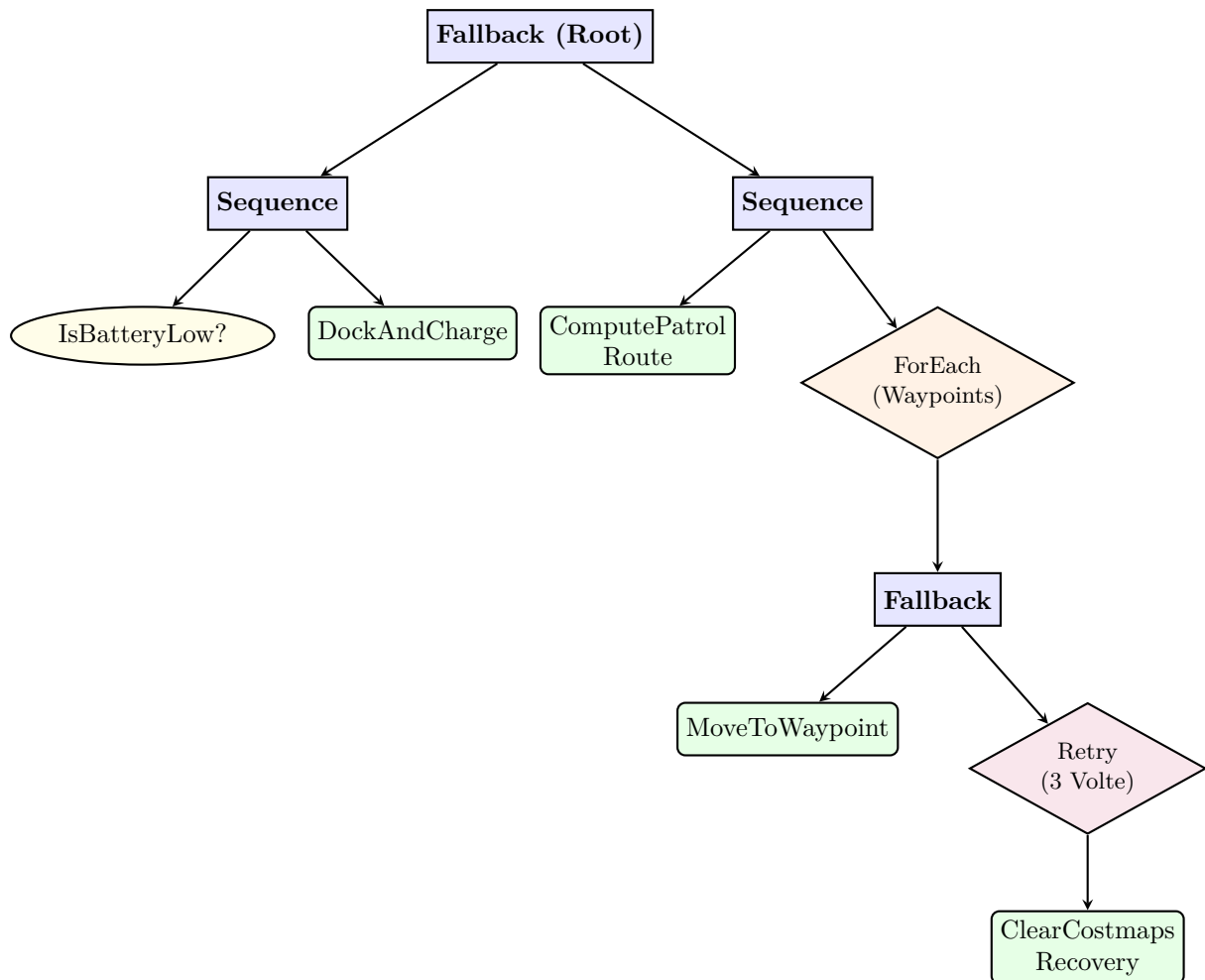


Figura 6: Behavior Tree gerarchico per la navigazione autonoma e la gestione delle anomalie di un robot di pattugliamento.

6.3.1 Analisi del ciclo di esecuzione (*Trace dell'Albero*)

Quando il sistema è in esecuzione, il selettore principale (**Fallback Root**) invia periodicamente il Tick verso il basso:

1. **Ispezione della Sicurezza:** Il Tick entra per primo nel ramo sinistro (**Sequence Batteria**). La condizione **IsBatteryLow?** analizza lo stato termico e di carica.
 - *Caso A (Batteria Carica):* La condizione fallisce, restituendo immediatamente **FAILURE**. La **Sequence** si interrompe. Il padre **Fallback Root** vede un fallimento sul ramo sinistro e, per la sua natura algoritmica, **passa immediatamente al ramo destro** per mantenere il robot operativo.
 - *Caso B (Batteria Scarica):* La condizione ha successo (**SUCCESS**). La **Sequence** prosegue ed emette il Tick verso l'azione **DockAndCharge**. Questa azione invia un comando di navigazione alla base di ricarica tramite un'Azione ROS 2 e restituisce **RUNNING**. Il ciclo si blocca qui finché il robot non è in carica, impedendo l'esecuzione del pattugliamento.
2. **Esecuzione del Task Principale:** Nel caso A, il Tick fluisce nel ramo destro (**Sequence Patrol**). Viene invocata l'azione **ComputePatrolRoute** che calcola i punti di ronda sulla mappa. Una volta ottenuti (**SUCCESS**), il ciclo entra nel decoratore iterativo **ForEach**.
3. **Gestione Robusta delle Collisioni (Recovery Wrapper):** Per ogni singolo waypoint, il **Fallback** sottostante tenta l'azione **MoveToWaypoint**.
 - Se il robot viaggia senza ostacoli, l'azione restituisce **RUNNING** e poi **SUCCESS** all'arrivo. Il **Fallback** si ritiene soddisfatto e passa al punto successivo.
 - Se un ostacolo imprevisto (es. una persona ferma nel corridoio) blocca il percorso, **MoveToWaypoint** fallisce (**FAILURE**). Il **Fallback** non fa fallire l'albero, ma dirotta il Tick sul ramo destro di ripristino. Il decoratore **Retry** attiva l'azione **ClearCostmapsRecovery**, un servizio ROS 2 che pulisce la memoria locale dei sensori per eliminare eventuali letture fantasma. Questa operazione viene tentata fino a 3 volte. Se il ripristino ha successo, al ciclo successivo **MoveToWaypoint** potrà ricalcolare la traiettoria e completare il compito, garantendo un'autonomia totale sul campo.